In [5]:
```
import numpy as np
import pandas as pd
import re
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from gensim.models import Word2Vec
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D, Dense
from sklearn.model_selection import train_test_split
```

In [6]:
```
# 1. Load CSV
df = pd.read_csv("IMDB Dataset.csv")

# 2. Clean text: lowercase, remove HTML tags and punctuation
def clean_text(text):
    text = re.sub(r"<.*?>", " ", text)
    text = re.sub(r"[^a-zA-Z\s]", " ", text)
    return text.lower().split()

sentences = df["review"].apply(clean_text).tolist()
labels = df["sentiment"].map({"positive": 1, "negative": 0}).values
```

In [7]:
```
# 3. Split into training and testing data
X_train_text, X_test_text, y_train, y_test = train_test_split(
    sentences, labels, test_size=0.2, random_state=42
)

# 4. Train CBOW Word2Vec (sg=0 means CBOW)
cbow_model = Word2Vec(
    sentences=X_train_text,
    vector_size=100,
    window=5,
    min_count=2,
    sg=0,
    workers=4,
    epochs=5
)
```

In [8]:
```
    # 5. Represent each review by averaging its word embeddings
def document_vector(words):
    vectors = [
        cbow_model.wv[word]
        for word in words
        if word in cbow_model.wv
    ]

    if len(vectors) == 0:
        return np.zeros(100)

    return np.mean(vectors, axis=0)

X_train = np.array([document_vector(s) for s in X_train_text])
X_test = np.array([document_vector(s) for s in X_test_text])
```

In [10]:
```
# 6. Build neural network for sentiment classification
model = Sequential([
    Dense(64, activation="relu", input_shape=(100,)),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

In [11]:
```
# 7. Train model
model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.2
)

# 8. Evaluate on test data
loss, accuracy = model.evaluate(X_test, y_test)
```

```
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
Epoch 1/5
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8024 - loss: 0.4358 - val_accuracy: 0.8319 - val_loss: 0.3847
Epoch 2/5
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.8321 - loss: 0.3766 - val_accuracy: 0.8366 - val_loss: 0.3698
Epoch 3/5
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.8396 - loss: 0.3676 - val_accuracy: 0.8406 - val_loss: 0.3614
Epoch 4/5
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8425 - loss: 0.3611 - val_accuracy: 0.8428 - val_loss: 0.3581
Epoch 5/5
1000/1000 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.8428 - loss: 0.3595 - val_accuracy: 0.8461 - val_loss: 0.3558
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8457 - loss: 0.3596
Test Loss: 0.35955318808555603
Test Accuracy: 0.8457000255584717
```

In [ ]: